

GSoC'26 Project Proposal

Learning Unlimited: Theming Improvements

Oval17: Anurag Singh



1. Basic Details

Name:

Anurag Singh

Contacts & Information:

Email: anuragsingh7051@gmail.com

GitHub Profile: [Oval17](#)

Twitter: [Oval](#)

Location: Jaipur, Rajasthan, India

Time zone: Indian Standard Time (UTC+5:30)

Education Details:

Currently, I am in my last year of academic pursuit, enrolled in a double major program encompassing Computer Science and Physics at **BITS Pilani**.

2. Share Links, of your previous work on opensource projects.

For the past 17-18 months, I have actively engaged in contributing to open-source projects. Along this journey, I have acquired valuable knowledge spanning embedded systems, system design, object-oriented programming, machine learning, Arduino, and Raspberry Pi.

Additionally, I have tackled numerous algorithmic challenges on platforms such as Codeforces and LeetCode, nurturing a keen aptitude for programming and problem-solving in a broader context.

I have worked with SugarLabs in my 2nd Year under GSoC '24 and completed one of their flagship project which was to bring Sugar OS on RPi : [Details](#)

Projects/Repository/Pull Requests Links	Description
Konva-Drawboard	A simple and interactive drawing board built with React and Konva.js.
Point-Cloud Data Viewer	A modern 3D point cloud visualization tool built with React, TypeScript, and Three.js
Advanced Staggared View	A React TypeScript application that demonstrates advanced image loading with service worker caching and staggered loading techniques.
Compilation of my C projects	I keep working on different C projects in my Hobby time to get deeper understanding of systems
Tiled	Tiled is a general purpose tile map editor for all tile-based games, have been contributing to it.

I have made over **650+ contributions** on GitHub, including more than **100+ pull requests** and involvement in over **60+ issue resolutions**, both active and resolved. My contributions have extended across various organizations and projects.

I have a lot of **Frontend Experience**, which is most suitable for this project.

3. Convince us that you will be a good fit for this project, by sharing links to your contribution to Learning Unlimited

During the past weeks, I've been an active member of the Learning Unlimited community, immersing myself in its ethos and methodologies. Throughout this period, I've gained substantial insights into the codebase and community, actively contributing to various activities and diligently addressing numerous issues, bugs, discussions and different aspects of projects overall.

This journey has been a profound learning experience, enriching my understanding of Learning Unlimited core functionalities, and a deep understanding of different aspects of codebase and fostering my growth within the community.

Here are the pull requests I have generated for Learning Unlimited:

Pull Request Link	Description	Status
-------------------	-------------	--------

#4201	Added pytest-django + pytest-xdist for parallel test execution improving Performance by almost 40 percent	Merged
#4237	Fix/email status button overflow on text droplets	Merged
#4234	Fix npm installation: re-add npm to packages_base.txt	Merged
#4205	Hide common searches for user types not available in the program	Merged
#4016	Bypass secure_required HTTPS redirect when DEBUG is True	Merged
#4015	Pre-select 'Student' in user selector when viewing student schedules.	Merged
#4004	Autofill nametag title with group name when 'Other group' is selected.	Merged
#4002	Fix theme editor formatting: move closing divs outside circles-only.	Merged
#4001	Fix extra b's in LaTeX output by decoding bytes to str	Merged
#4000	Add tabindex to class flag create/edit form elements	Merged
#3999	Rename Phase Zero to Program Lottery in user-facing text.	Merged
#4204	Fix Theme Editor Crash when Images/themes contains subdirectory	Open
#4007	Fix MessageRequest admin: add blank=True to processed_by and sendto...n_name	Open
#4209	Upgrade Knockout.js 2.3.0 → 3.5.1 and Select2 3.4.3 → 4.1.0	Work In Progress

Made over **20+ Quality Pull Requests and** thoroughly resolved Issues, got them reviewed and merged in the codebase and got a very deep understanding of the codebase.

Additionally, I've tested and reviewed over 20+ **Issues and PRs.**

Created a very comprehensive list of Draft PRs that acted as a resource for generating test cases and helped other contributors understand the codebase.

Here are the Issues, Reviews and discussion I have Contributed to in **Learning Unlimited:**

Issue Link	Description	Status
#3963	Printables have encodings.	Closed
#3941	Commonly used searches should only list those for relevant user types	Closed
#3940	Theme editor formatting messed up on circles	Closed
#3931	Nametags "title of ids" field should autofill better when "other group" is selected.	Closed
#3917	Review pytest-django for test execution and parallelization.	Closed
#3975	Fix issue installing npm	Closed
#1358	Add HTML tabindex on class flag create/edit form.	Closed
#3786	Upgrade knockoutjs and select2.	Open
#1983	Can't edit MessageRequest from the admin panel	Open
#1764	Theme editor chokes if there is a directory under images/theme	Open
#1734	Can't test finaid app on dev server.	Closed
#4378	Reviewed PR	Closed
#4367	Reviewed PR	Closed
#4331	Reviewed PR	Closed
#4007	Resolved and Reviewed PR Comments	Closed
#4017	Contributed to Discussion by addressing and discussing different concerns	Discussion
#4140	Contributed to Discussion by addressing and discussing different concerns	Discussion

I've also engaged with matters related to Learning Unlimited, assisting with running it in different environments and addressing related issues.

I have understood different parts of codebase, starting from very basics and understanding the whole flow architecturally, I have already invested around 100+ hours in the last few weeks, and I will keep on contributing for the upcoming weeks and months.

4. Project Details

Name : Theming Improvements

Detailed description:

Our website's theming system, which allows individual chapters to customize the look of their websites, is due for a major overhaul. The site currently uses Bootstrap 2.3.2, which is over a decade old the current version is 5.3. Bootstrap is used minimally (primarily for buttons and minor elements), and there are existing issues with color contrast. This project involves upgrading Bootstrap to the latest version, expanding its use throughout the website for standardized styling, and implementing Bootswatch themes as default customization options. Bootswatch would provide cohesive color sets, fonts, and other stylings that administrators could select and then tweak appropriately to work for their chapters.

Expected size: 350 hours

Possible mentors: Katherine Brumberg, Will Gearty

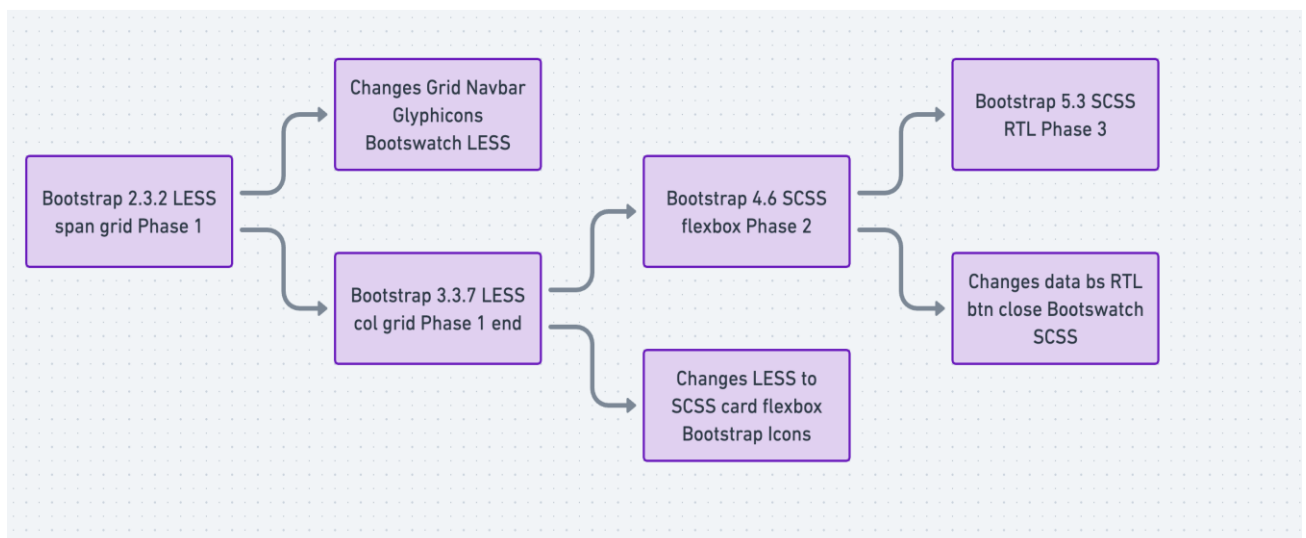
This project comprises **Four** crucial components:

Part 1: Bootstrap Migration Pipeline (2 → 3 → 4 → 5)

1.1: Why sequential, not direct jump to BS5?

> Mentor @willgearty confirmed this in discussion: *"We never need to support more than one version of Bootstrap. Each step would probably be one or more separate PRs."* Direct migration from BS2 → BS5 would be a massive untestable diff. The sequential approach means each PR is reviewable, each step is testable in isolation, and live chapter sites can deploy at any phase boundary safely.

Bootstrap Migration Roadmap



1.2: Current Bootstrap 2 usage audit (from my codebase reading):

Bootstrap 2 Pattern	Where Used	Bootstrap 5 Equivalent
.span* / .row-fluid	All theme templates, registration forms	.col-* / .row
.navbar-inner, .nav-collapse	droplets, bigpicture navbars	.navbar-expand-lg, .navbar-collapse
.btn-navbar	droplets admin bar	.navbar-toggler
.icon-* (sprite icons)	Shared templates, buttons	Bootstrap Icons (font)
.hero-unit	bigpicture index.html	.jumbotron (BS3/4) → custom utility (BS5)
.well	Multiple form wrappers	card .card-body
data-toggle, data-target	Modals, dropdowns	data-bs-toggle, data-bs-target
.label-important	Class status badges	.badge .text-bg-danger

Part 2: LESS → SCSS Migration + Bootswatch Integration

Bootstrap 4 was the first version to drop LESS entirely in Favor of SCSS. This is the single most impactful infrastructure change in the project.

2.1 After migration target SCSS structure:

```

theme_data/
├── droplets/
│   └── scss/
│       ├── _variables.scss
│       │   └── $primary: #3a7abf;
│       ├── _bootswatch.scss
│       │   └── @import "bootswatch/cerulean/_variables";
│       └── main.scss
│           ├── @import "bootstrap/scss/bootstrap";
│           ├── @import "esp_theme_variables";
│           └── @import "esp_theme_custom";

```

```

theme_data/
├── droplets/                                ← Priority theme
│   ├── less/
│   │   ├── variables.less                 (@primary-color: #3a7abf;)
│   │   └── main.less                     (.navbar { .navbar-inner { ... } })
│   ├── templates/
│   │   ├── navbar/
│   │   │   ├── nav.html
│   │   │   └── admin.html
│   │   └── main.html
│   └── scripts/
│       └── admin_bar.js
├── fruitsalad/                             ← MIT Splash default
│   ├── less/
│   │   ├── variables.less
│   │   ├── variables_optional.less
│   │   ├── main.less
│   │   └── splash_page.less              ← largest file (~300 lines)
│   └── templates/
├── bigpicture/
├── circles/
├── floaty/
└── barebones/                             ← migrated first (simplest)

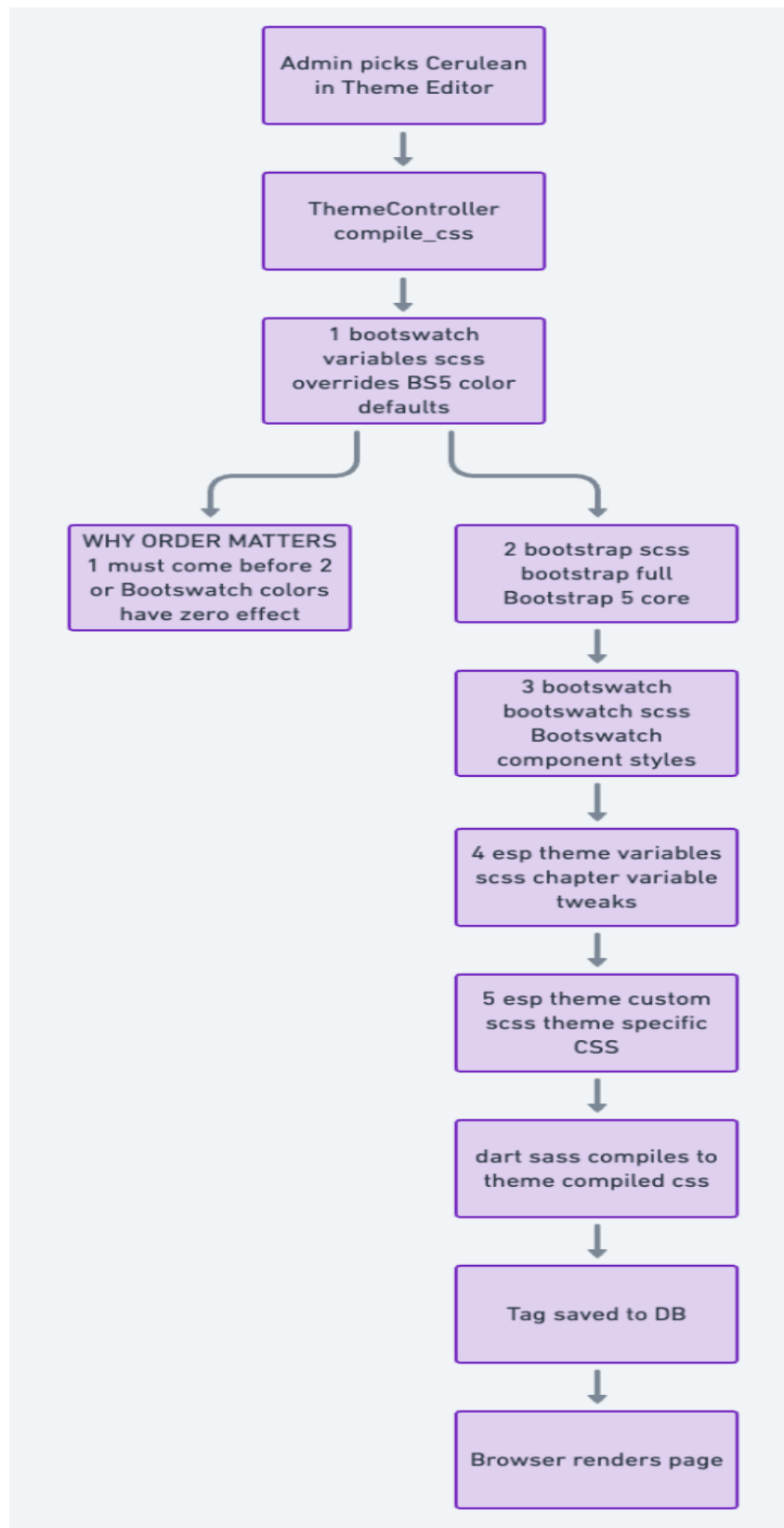
```

2.2 Current LESS theme structure (as I've read it from the repo):

2.3 LESS → SCSS syntax migration (the porting is mechanical but must be clean):

LESS	SCSS	Example
@variable: value;	\$variable: value;	@primary → \$primary
.mixin()	@include mixin()	clearfix() → @include clearfix()
#namespace > .mixin()	@use / @forward	Namespace imports
color: lighten(@primary, 10%)	color: lighten(\$primary, 10%)	Function calls identical
@import "variables"	@import "variables"	Same syntax

2.4 Bootswatch integration architecture:



2.5 Why this import order is critical:

Bootswatch variables.scss must come BEFORE Bootstrap core, otherwise Bootstrap's

own variable defaults take precedence and Bootswatch's color palette has no effect. This is a common mistake in Bootswatch integration that I've specifically researched and built around.

Note: 13 LESS files across 6 themes, this is completely tractable in 2 weeks and doing it manually (not with automated converters) produces clean, reviewable code.

Part 3: Bootstrap Usage Expansion + Accessibility

After the full Bootstrap 5 migration, the site still uses Bootstrap minimally. This component expands it into the shared templates that every chapter sees.

3.1 Current state (from reading the shared templates):

Registration forms, class catalogs, and admin pages have a mix of:

- Custom CSS classes with no semantic meaning (.class-card, .program-box)
- Tables with no Bootstrap styling - Modals with Bootstrap 2 data attributes
- No .form-control / .form-label on most inputs
- Inline styles (style="color: red")

3.2 Accessibility targets (WCAG 2.1 AA): -

- Color contrast ratio: 4.5:1 for normal text, 3:1 for large text
- aria-label on all icon-only buttons
- role="navigation" + aria-expanded on navbar toggles
- All form fields with <label> elements (not placeholder-only)

3.3 Target state:

It's given in Structure tree Below.

Bootstrap 5 Usage

Registration Flow

- └ form-control / form-label / form-select / form-check
- └ .progress (phase indicators)
- └ .badge (class counts)

Admin / Management

- └ .table.table-striped.table-hover
- └ .accordion
- └ .alert.alert-dismissible
- └ .modal (data-bs-* attrs)

Navigation (droplets)

- └ .navbar-toggler + .navbar-collapse
- └ .offcanvas
- └ .breadcrumb

Part 4 : Architecture Deep Dive: How the Theme System Works Today

This section demonstrates my actual understanding from reading the code, not just the project description. The full ThemeController pipeline (I've read every line of controllers.py):

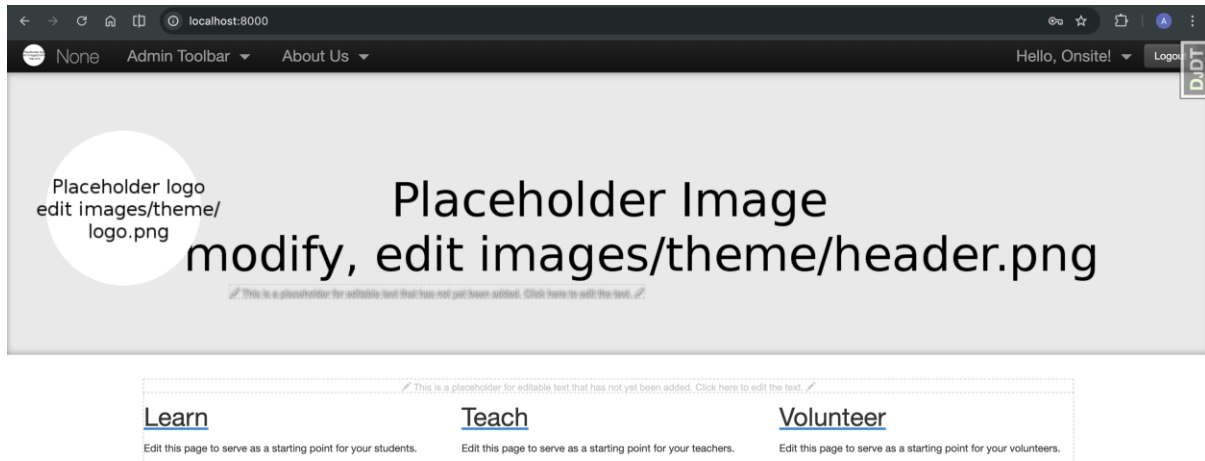
4.1 Why DB tag backward compatibility is non-negotiable:

- When a chapter pulls the new codebase, their DB still has the old "current_theme_params" tag e.g., {"primary_color": "#3a7abf"}.
- The variable name primary_color is defined in the theme's variables.less / variables.scss.
- Since I'm not changing variable names during the LESS→SCSS migration (only the prefix: @primary_color → \$primary_color), the existing DB tags continue to work.
- get_variable_defaults() reads the name without the prefix.

4.2 The Knockout.js dependency I'm already working on (PR #4209):

- The theme editor uses Knockout.js 2.3.0 (from 2013) and Select2 3.4.3 (from 2014) for its reactive UI. I am currently upgrading these in PR #4209 (Knockout.js 2.3.0 → 3.5.1, Select2 3.4.3 → 4.1.0).
- This directly unblocks the theme editor UI improvements in Phase 4, because Select2 4.x has the API needed to build a proper Bootswatch theme selector dropdown. This is an example of work I'm doing **beyond the stated project scope fixing** the underlying dependency that makes the project's UI goals achievable.

4.3 I have gone through each theme : (Just one E.g.)



Current state Bootstrap 2 in our droplets theme, for example: (live issues I found in the code can be seen):

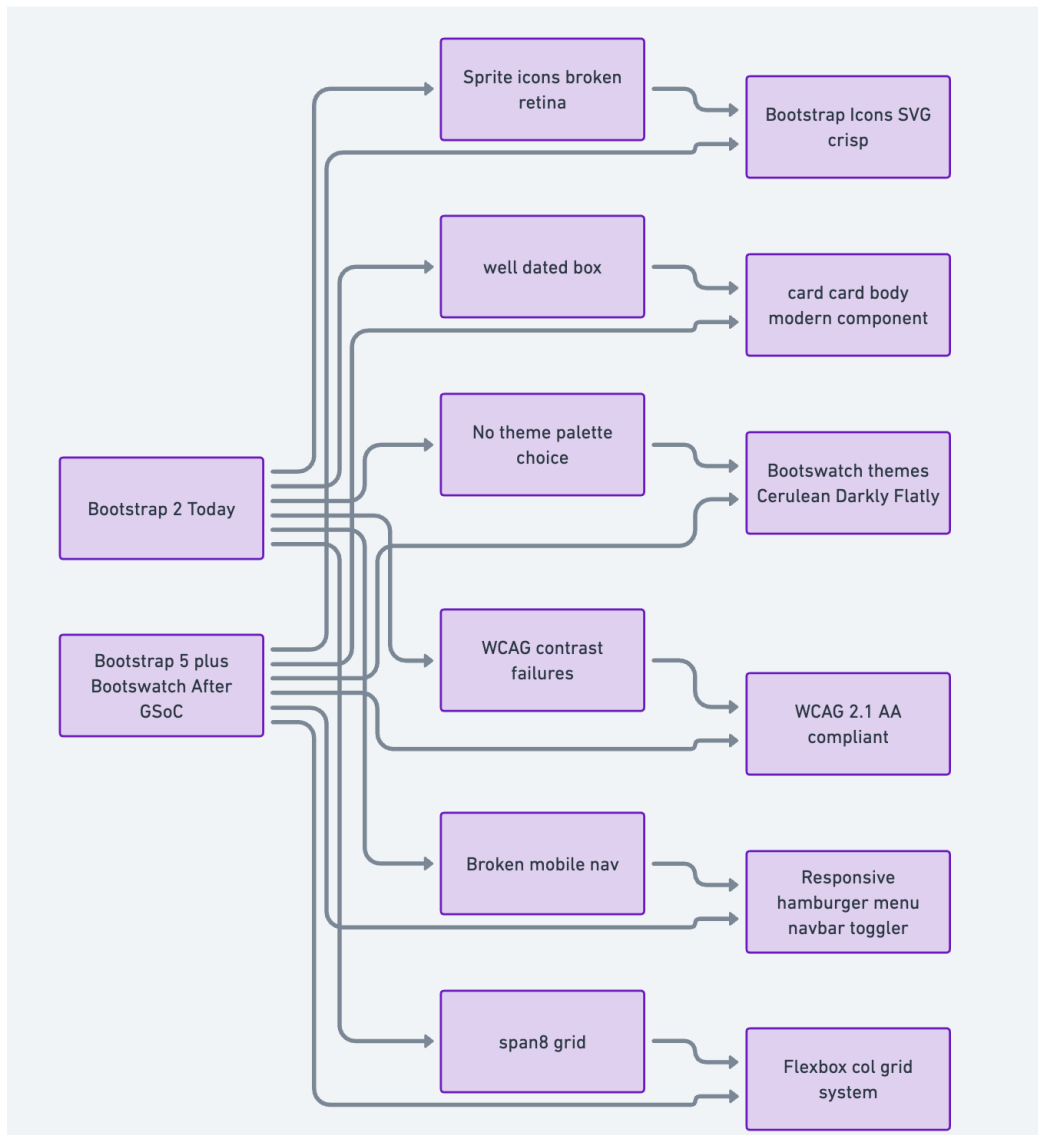
- .span8 / .span4 grid —> no flexbox, layout breaks on narrow screens
- In mobile view, it gets broken easily.
- .navbar-inner — removed in Bootstrap 3+, currently renders as an unstyled div.
- .icon-search, .icon-user — sprite-based icons, broken on retina/HiDPI displays.
- .well — deprecated component, visually dated.
- Fixed-width containers — causes horizontal scroll on mobile devices.
- Low color contrast on several text elements (WCAG 2.1 AA fail).

4.4 Testing Strategy: Testing is integrated into every phase and will include:

- Unit tests (extending esp/esp/themes/tests/)
- Existing pytest suite — the non-negotiable gate
- Visual QA (included in each PR description)
- Accessibility (Week 12, documented in Phase 5 PR)
- A detailed testing strategy for each phase can be found in the timeline.

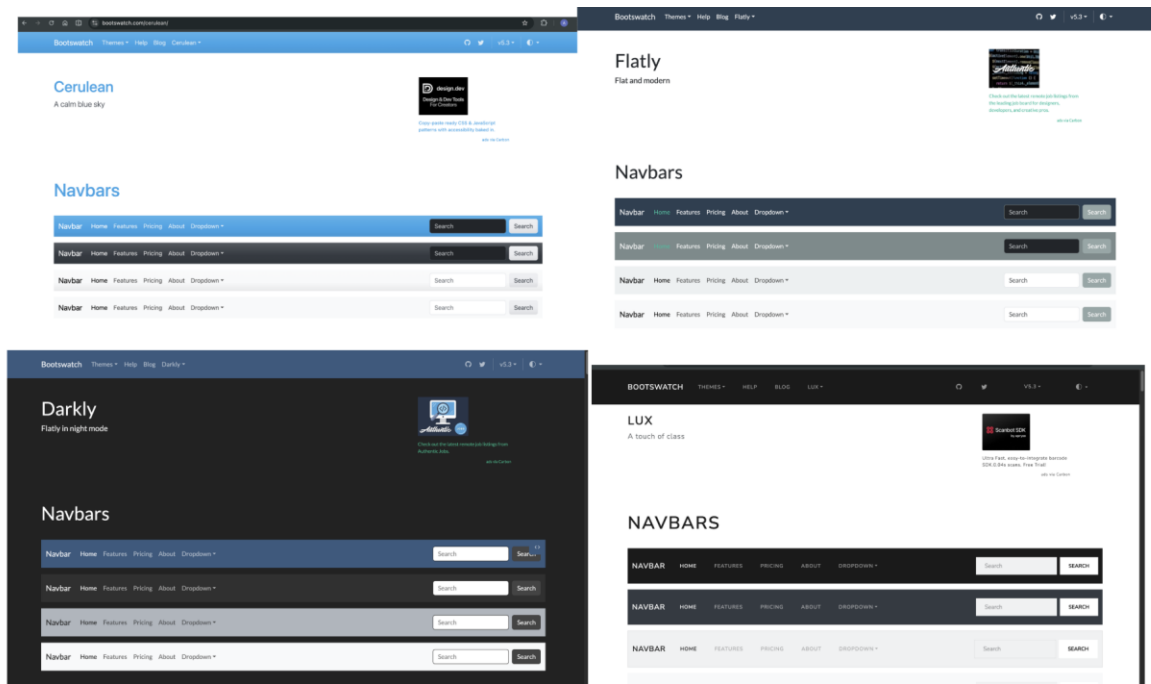
4.5 After Completion of all the parts, we will have

 Bootstrap 2 Today vs Bootstrap 5 Bootswatch After GSoC



4.6 Some Bootswatch design themes:

A depiction of how our themes will look after all the changes are made.



5.TimeLine:-

Break down the entire project into chunks and tell us what will you work on each week.

Weeks	Tasks to be Completed
Pre-GSOC (Apr – May)	- Complete PR #4209 (Knockout/Select2 upgrade). Finish feature/bootstrap-5-upgrade PR review. - Audit every Bootstrap 2 class across all templates and document in a mapping table on the wiki. - Study Bootstrap 3/4/5 changelogs in full.
Community Bonding Period (1st May – 24th May)	- Finalize migration plan with mentors. Align on droplets design direction. - Set up local environment with dart-sass. - Confirm CI setup for each phase. - Discuss Bootswatch theme selector UX with Mentors.
Week-1 (25th May – June 2nd) (Phase – 1)	- Bootstrap 2→3: Update package.json BS dependency. - Port barebones and droplets (priority) to BS3. - Migrate .span → .col .row-fluid → .row, .navbar-inner → .navbar.

Week-2 (June 3rd – 9th) (Phase - 1)	- Port `bigpicture`, `circles`, `floaty` to Bootstrap 3. - Add Bootswatch 3 (LESS) optional integration to `ThemeController`.
Week-3 (June 10th –16th) (Phase – 1)	- Port fruitsalad. - Write unit tests: <code>has_scss()</code>, <code>compile_css()</code> with BS3, <code>get_variable_defaults()</code> round-trip, Bootswatch 3 LESS drop-in. - Run full pytest suite - all 756 tests must pass. - Submit Phase 1 PR with before/after droplets screenshots at 375px/768px/1280px.
Week-4 (June 17th – 23rd) (Phase – 2)	- Bootstrap 3→4 + LESS→SCSS: Update package.json (Bootstrap 4.6, sass ^1.70.0). - Wire `_compile_css_scss()` as default. Port `droplets` and `barebones` LESS→SCSS.
Week-5 (June 24th – 30th) (Phase – 2)	Port `bigpicture`, `circles`, `floaty` LESS→SCSS. Replace Glyphicons with Bootstrap Icons 1.x.
Week-6 (July 1st – 7th) (Phase – 2)	- Port fruitsalad LESS→SCSS (<code>splash_page.less</code> → <code>_splash_page.scss</code>). BS4 component updates (<code>.card</code>, <code>.jumbotron</code>). - Write unit tests: <code>_compile_css_scss()</code> with BS4, <code>\$variable</code> parsing in <code>get_variable_defaults()</code>, DB tag round-trip (old LESS params → new SCSS compiler produces valid CSS without error). - Run full pytest suite. - Submit Phase 2 PR.
Week-7 (July 8th – 14th) Midterm Evaluation	- Bootstrap 4→5: Update package.json. Grep-audit `data-toggle`/`data-target`/`data-dismiss` → `data-bs-*` across all templates. - Update `droplets/admin_bar.js` for BS5 JS. - Run pytest suite confirm no regressions.
Week-8 (July 15th –21st) (Phase – 3)	- BS5 CSS changes (<code>.jumbotron</code>, <code>.btn-close</code>, <code>.text-start</code>). Bootswatch 5 integration, <code>get_bootswatch_themes()</code>, theme selector UI. - Write unit tests: <code>get_bootswatch_themes()</code> returns correct list, SCSS compile with Bootswatch import order, Bootswatch theme selector persists to DB tag correctly. <code>config_form.py</code> updates & Cache Busting

	• Run pytest suite. • Submit Phase 3 PR.
Week-9 (July 22nd – 28th) (Phase – 4)	• Bootstrap usage expansion: registration flow, `form-control`, `form-label`, `form-select`, `form-check`. • Class catalog `.card` components. `.progress` bars. • Manual QA: walk through student registration flow end-to-end on local dev.
Week-10 (July 29th – Aug 4th) (Phase – 4)	• Admin pages: `.table`, `.accordion`, `.alert.alert-dismissible`. Modal `data-bs-*` updates. • Manual QA: admin creates program, manages modules, sends email - verify no visual regressions.
Week-11 (Aug 5th – Aug 11th) (Phase – 4)	• Navigation: `.navbar-toggler`, `.offcanvas` mobile sidebar. Breadcrumbs. Run pytest suite — full pass. Mobile QA: Chrome DevTools at 375px/768px for registration flow + admin dashboard. Submit Phase 4 PR with mobile screenshots.
Week-12 (Aug 12th – Aug 18th) (Phase – 5)	• Accessibility audit: axe-core scan on home page, registration flow, class catalog, admin dashboard for each of 6 themes + 3 Bootstrap themes. • Fix all WCAG 2.1 AA failures. Document results per page per theme in PR.
Week-13 (Aug 19th – Aug 25th) (Phase - 5)	• Documentation: Bootstrap migration guide (wiki), SCSS theme creation guide, Bootstrap integration guide. • Final PR cleanup, address all review feedback. Blog post. • Final pytest run 756+ tests and more test on demand
Final Evaluation	• All 5 phases delivered. 6 themes on Bootstrap 5, Bootstrap integrated, accessibility audit passed, full test suite green, documentation complete.

6. Some Important Questions:-

How many hours will you spend each week on your project?

During my summer vacation, I anticipate being able to dedicate approximately 60-70 hours per week to GSoC-related activities. I am fully prepared to devote most of my time to GSoC.

How will you report progress between evaluations?

I will maintain an active presence on GitHub by regularly submitting pull requests. Additionally, **I will actively engage on social media platforms such as LinkedIn, Instagram, and Twitter, where I will share updates on my progress, interact with the community, and promote participation in the project.**

How Will I Handle Stretch Goals (if ahead of schedule) ?

1. Theme live preview in the theme editor: an AJAX endpoint that compiles SCSS on-demand and injects CSS via <style> tag without a full page reload. High-value UX for chapter admins.
2. Droplets visual redesign, @willgearty mentioned redesigning droplets as a stretch goal. With Bootstrap 5's flexbox utilities, a modern redesign is achievable within the project scope.
3. Automated accessibility CI check Add pa11y-ci to GitHub Actions to block merges that introduce WCAG AA failures. Permanently protects accessibility.

Discuss your post-GSoC plans. Will you continue contributing to Learning unlimited after GSOC ends?

My primary post-GSoC plan involves staying active in the ESP-Website community as a maintainer-level contributor. The theming system, once on Bootstrap 5, will need ongoing maintenance as Bootstrap releases minor updates. I want to own that. I am also interested in continuing the Knockout.js upgrade work (PR #4209) and potentially proposing the "Better Onsite Management" or "New Module Management UI" features in future cycles.

This is not a one-summer contribution. Learning Unlimited is genuinely useful software serving tens of thousands of students every year, and I want to keep making it better and pursue a very long-term vision and contribution with the org.

Discuss all AI Disclosure Per the ESP-Website GSoC AI disclosure requirement ?

Tools used: Claude (Anthropic) used for understanding core parts of codebase and to

dive deeper in codebase

Scope: Proposal text drafted with AI assistance, then reviewed, verified, and edited by me personally.

Human verification: All technical content (file paths, class names, DB tag names, migration mappings, ThemeController method names) was verified by me directly against the codebase.

7. Appendix: Key Files I've Read Thoroughly : -

1. esp/esp/themes/controllers.py — ThemeController, all ~800 lines
2. esp/esp/themes/theme_data/droplets/ -> priority theme, all files
3. esp/esp/themes/theme_data/fruitsalad/ -> MIT Splash, all LESS files
4. esp/esp/themes/settings.py -> path configuration
5. esp/public/media/theme_editor/package.json -> npm dependency chain
6. Discussion #4017 mentor's stated architecture decisions -> Issue #3808, #3809, #3810
7. sub-issue structure and requirements -> esp/esp/tagdict/models.py
8. Tag model -> (backward compatibility requirement source)



Learning Unlimited

Thankyou !!

Proposal by Anurag Singh (Oval17) | GSoC 2026 | Learning Unlimited / ESP-Website